`com.polarbookshop.orderservice.order.event` package in your Order Service project (order-service).

Listing 10.22 A DTO representing the event about orders being accepted

```
package com.polarbookshop.orderservice.order.event;

public record OrderAcceptedMessage (
  Long orderId
){}
```

We can bridge the REST layer with the stream part of the application using a `StreamBridge` object that allows us to send data to a specific destination imperatively. Let's break this new functionality down. First, we can implement a method that accepts an `Order` object as input, verifies it's accepted, builds an `OrderAcceptedMessage` object, and sends it to a RabbitMQ destination using `StreamBridge`.

Open the `OrderService` class, autowire a `StreamBridge` object, and define a new `publishOrderAcceptedEvent` method.

Listing 10.23 Implementing the logic for publishing events to a destination

```
package com.polarbookshop.orderservice.order.domain;

import com.polarbookshop.orderservice.book.BookClient;
import com.polarbookshop.orderservice.order.event.OrderAcceptedMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.stream.function.StreamBridge;
import org.springframework.stereotype.Service;
...

@Service
public class OrderService {
  private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);

  private final BookClient bookClient;
  private final OrderRepository orderRepository;
```